

Exercise 1: Analyzing Session Management Practices

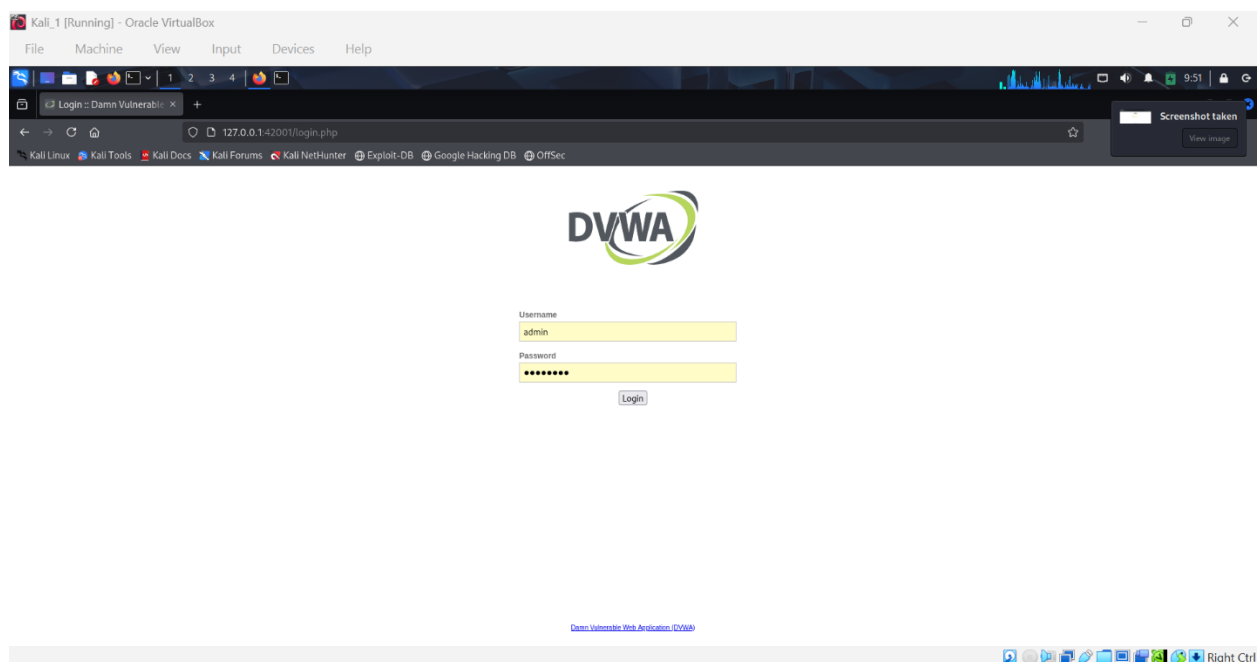
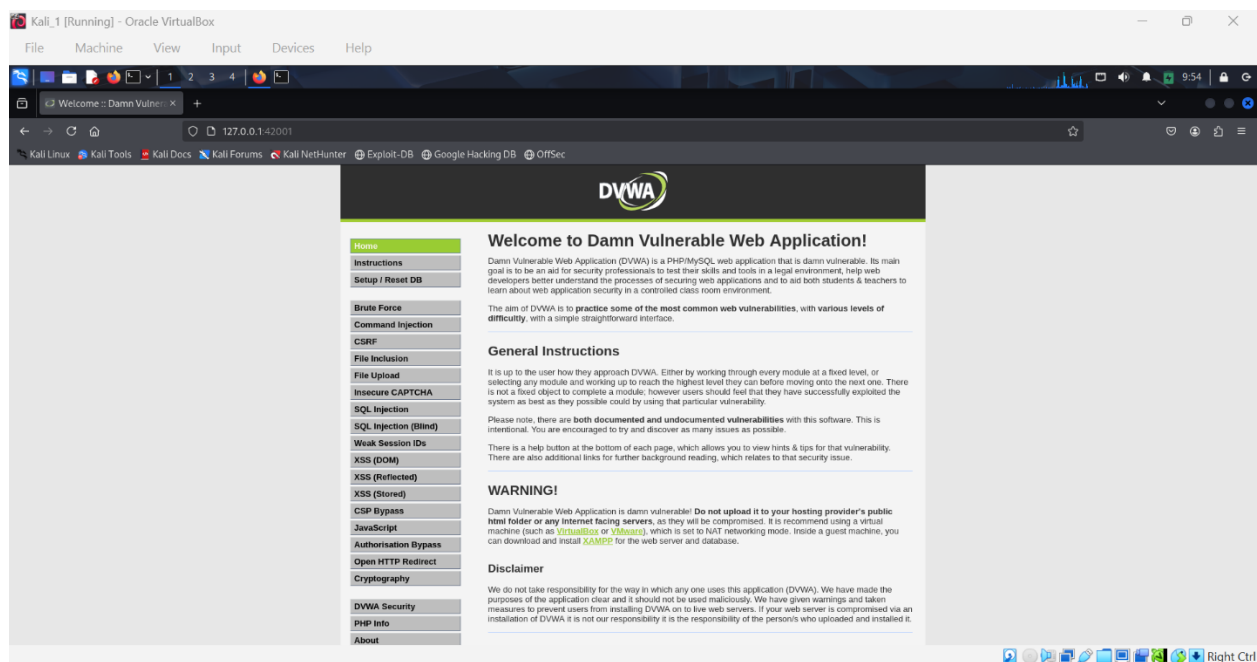


Figure 1 – Login into dvwa login using the default username and password



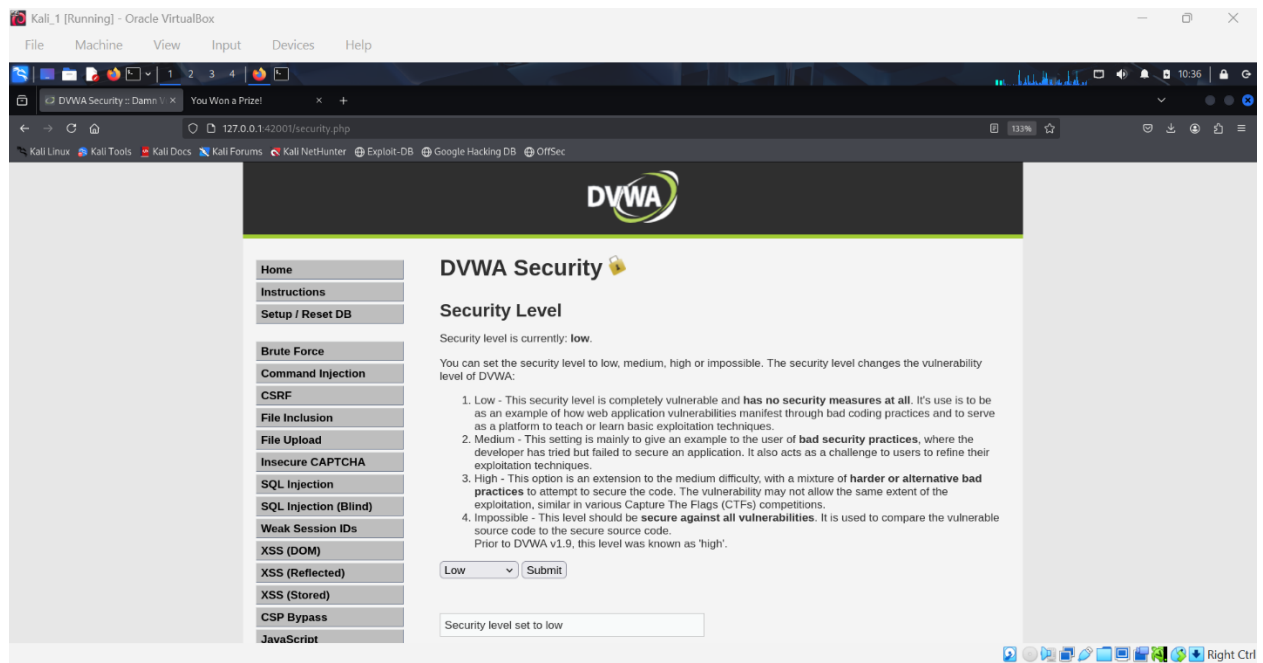


Figure 2 – Set the dvwa security level to low and submit

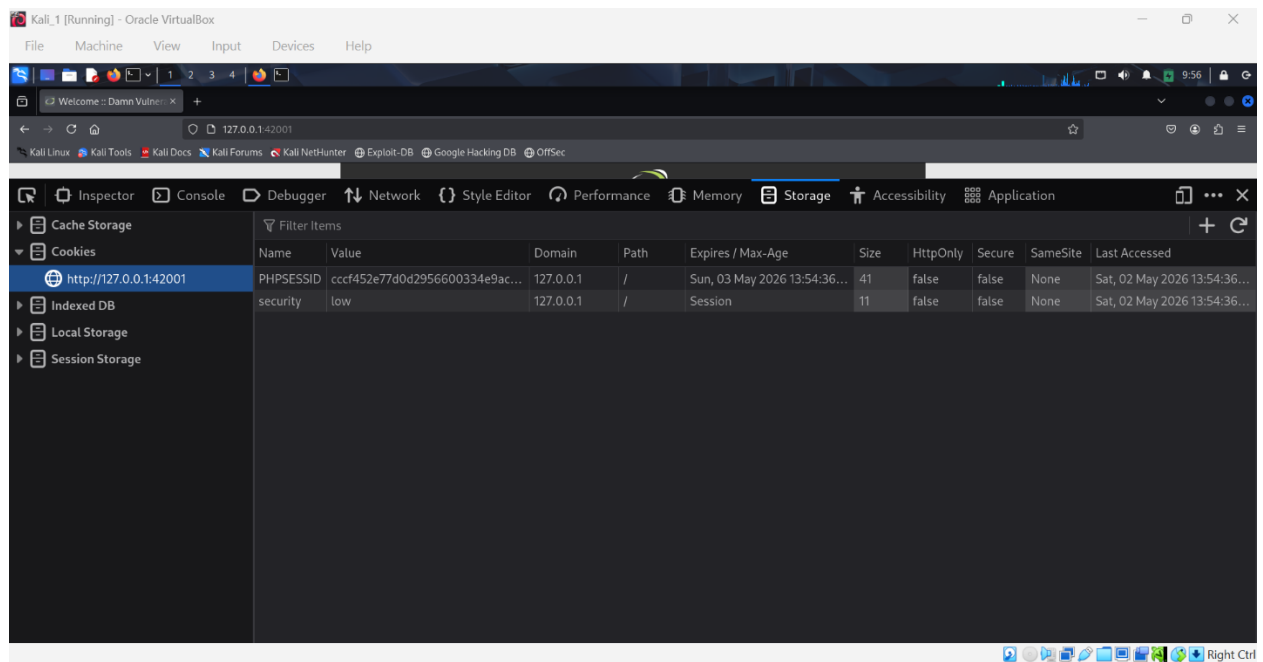


Figure 3 – Press F12 on the keyboard to open the Developer Tools, go to the Storage tab and Click on Cookies

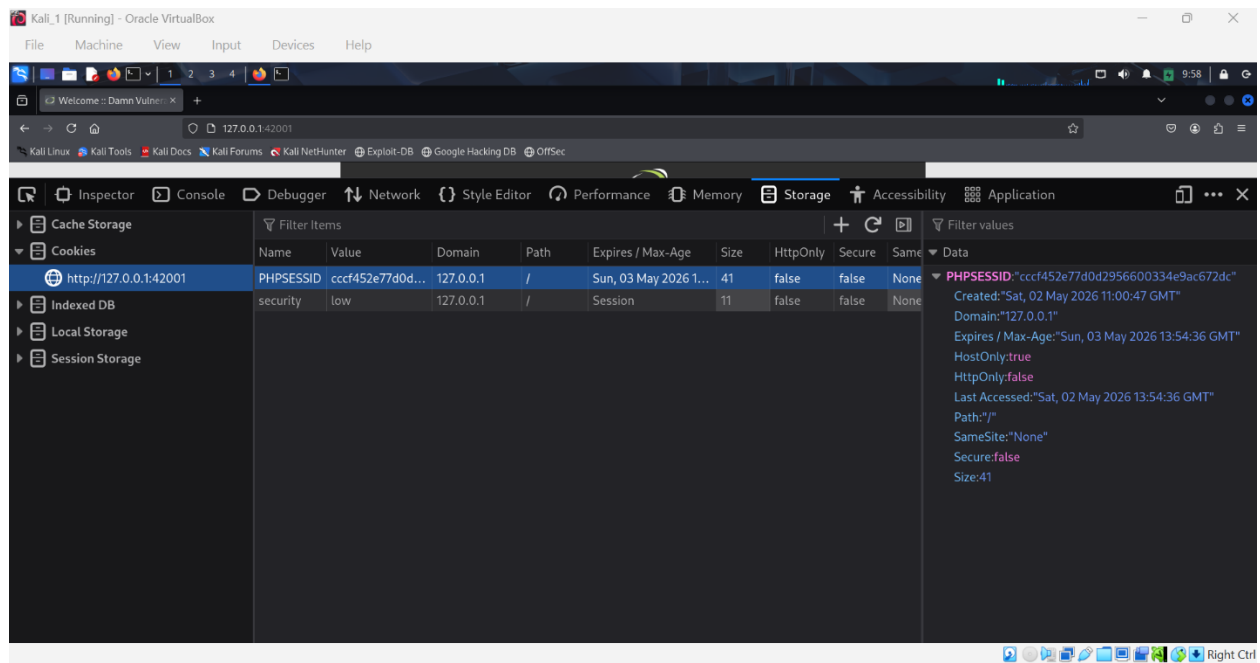


Figure 4 – On the cookie, click on PHPSESSID and notice the following cookies attributes

- HttpOnly is false
- SameSite is None
- Secure is false

For HttpOnly is false, this means that Cross site Scripting (XSS) attacks can be carried out.

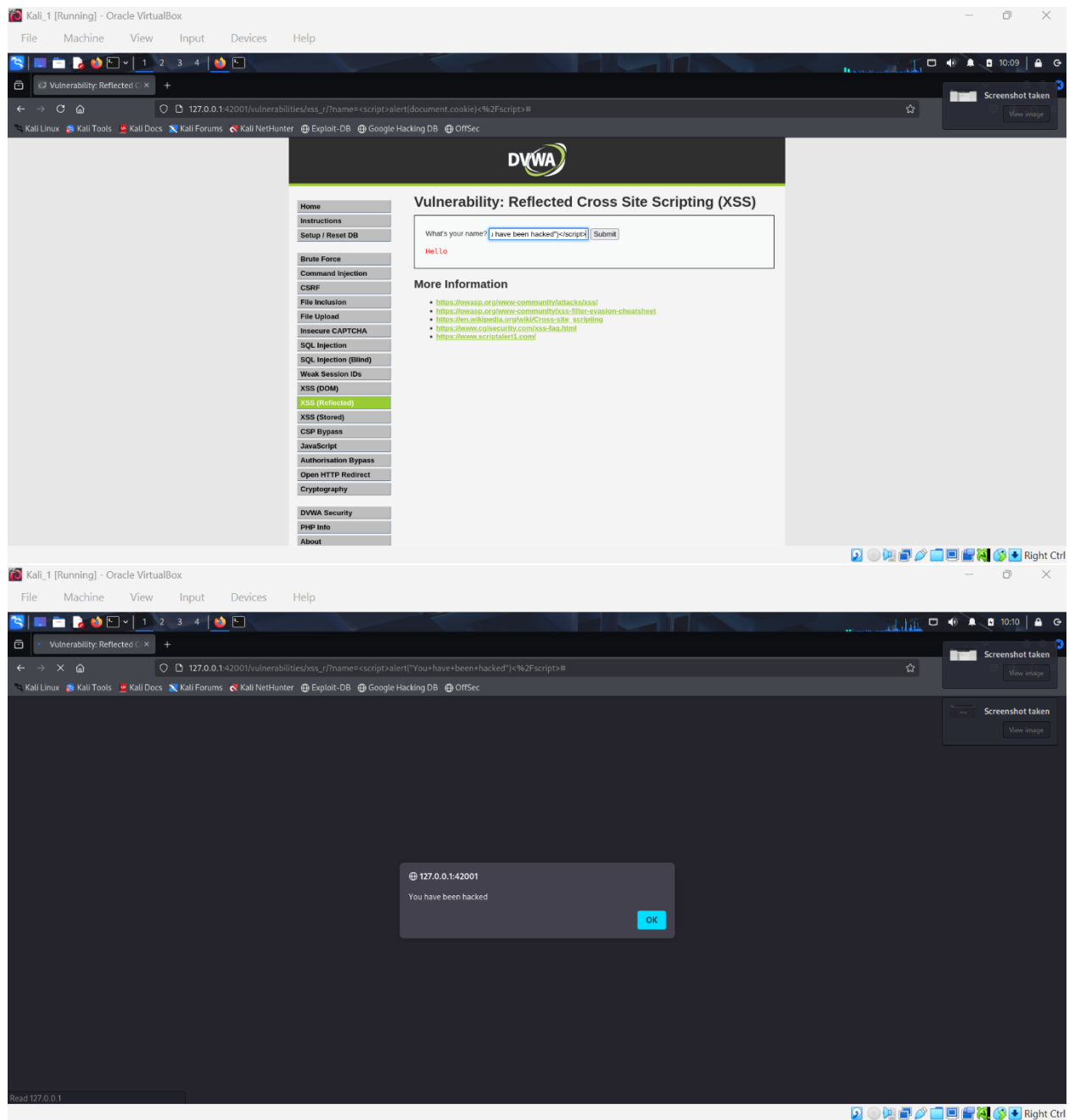


Figure 5 – A simple code like `<script>alert("You have been hacked") </script>` shows how malicious attacks can be carried out

For Secure is false, this means data is transmitted in plain text, making it vulnerable to interception by attackers

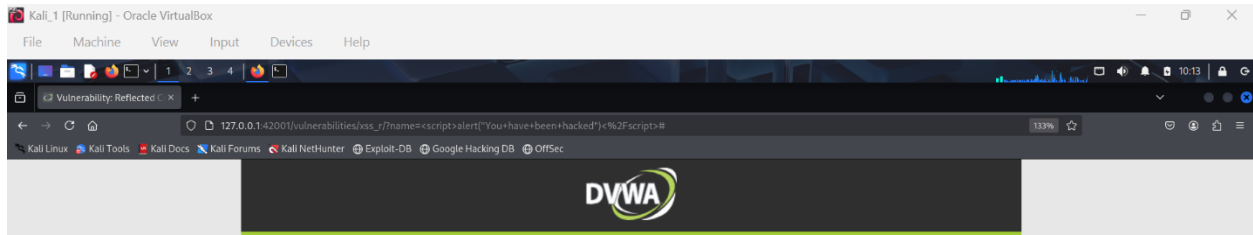
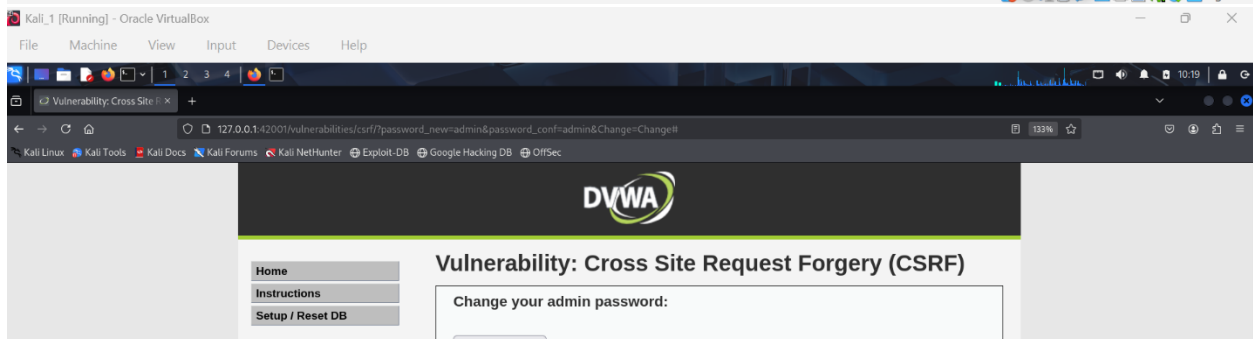
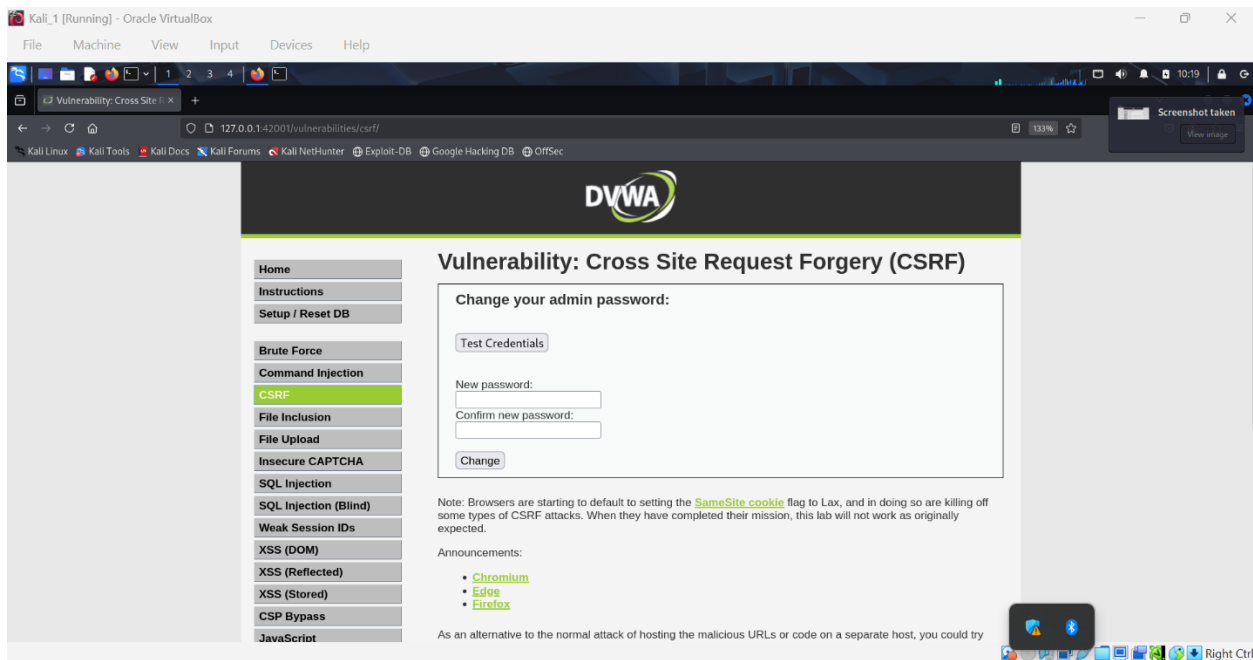


Figure 6 - The code `<script>alert("You have been hacked") </script>` which was inputted in the input field and submit was shown on the cookie header as plain text instead on be encrypted.

SameSite is false, this means that Cross-Site Request Forgery (CSRF) attacks can be carried out



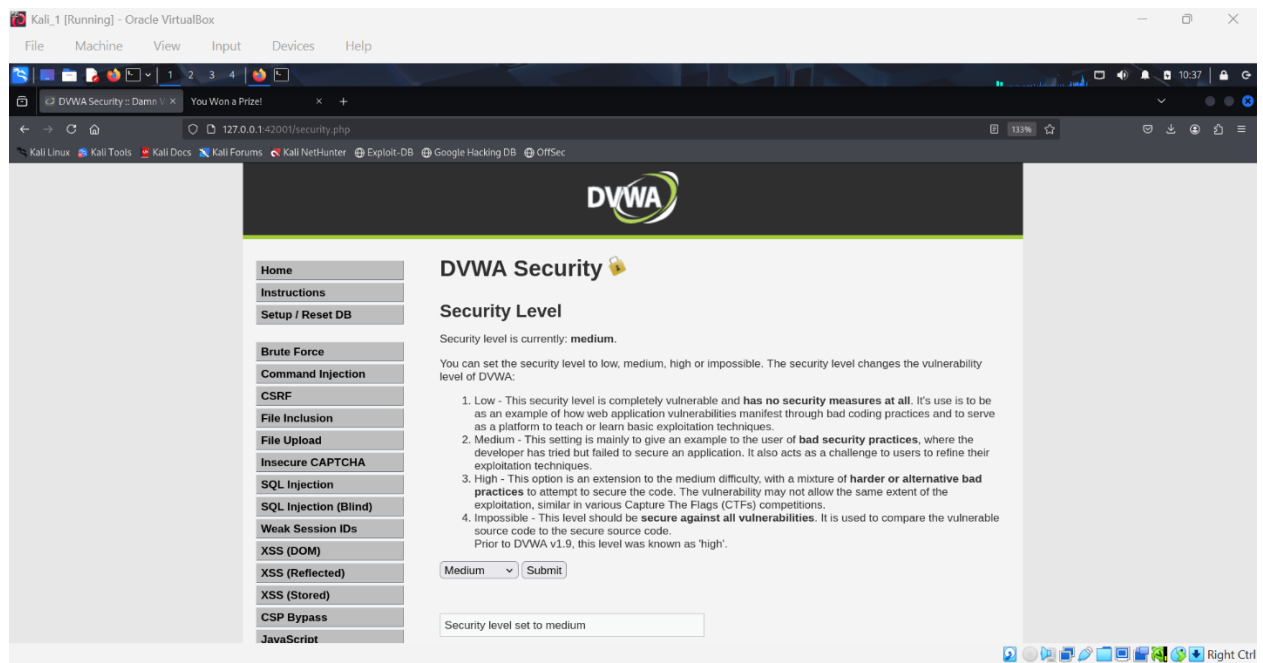


Figure 7 - If the dvwa security is set to medium

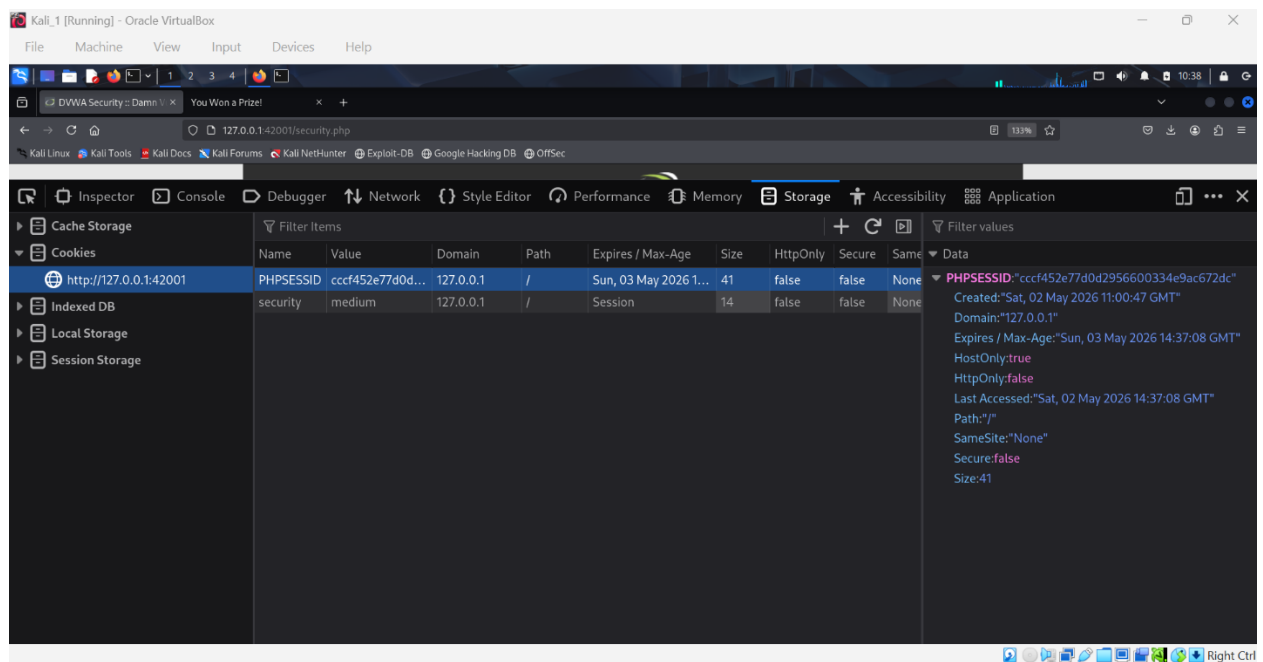


Figure 8

- HttpOnly is false
- SameSite is None
- Secure is false

Meaning Cross site Scripting (XSS) attacks can still be carried out.

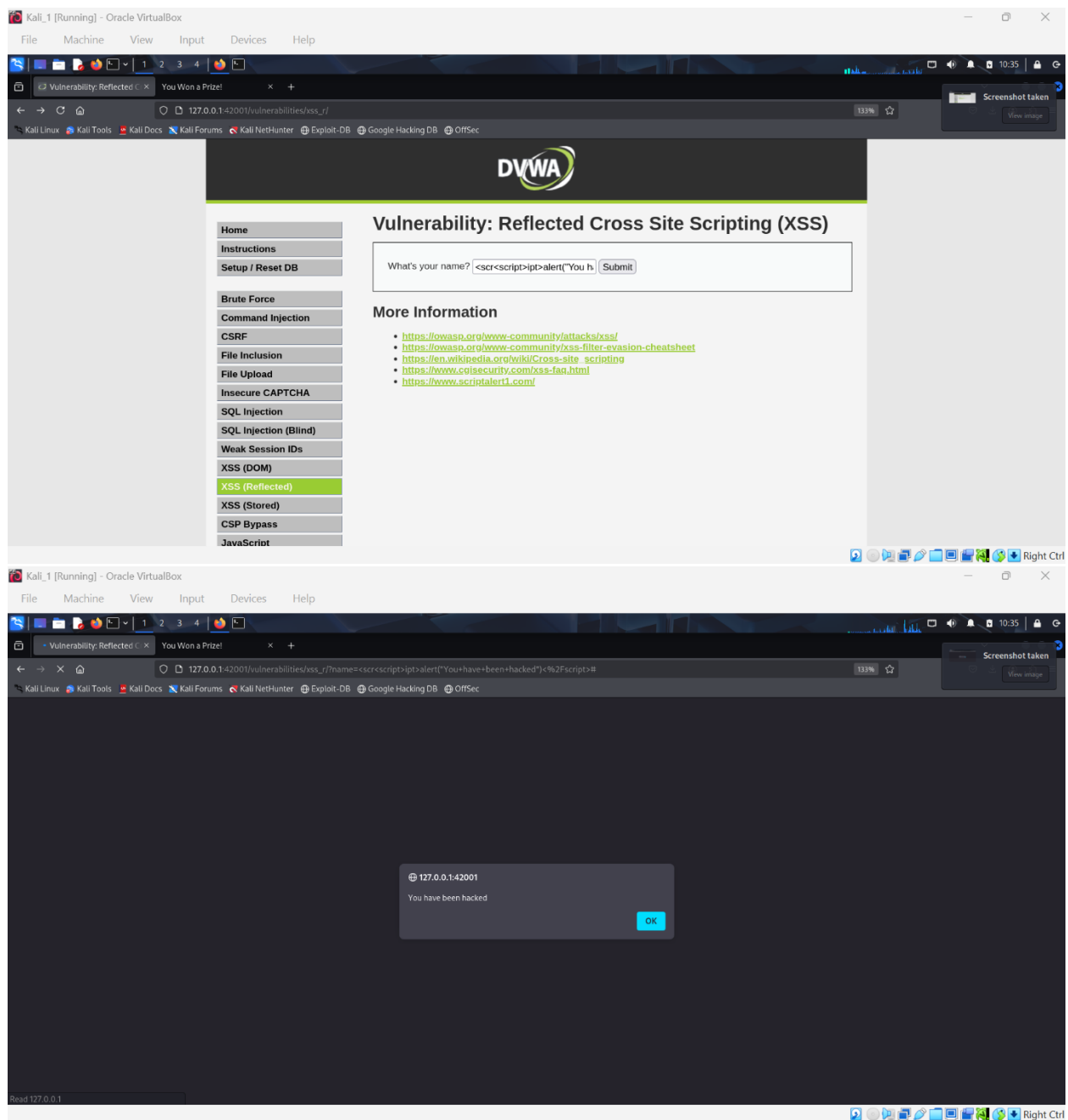


Figure 9 – A simple code like `<scr<script>ipt>alert("You have been hacked")</script>` shows how malicious attacks can be carried out

FINDINGS

The following session cookie was observed after login:

- Cookie Name: PHPSESSID
- Value: [randomly generated alphanumeric string, e.g., a1b2c3d4e5f6...]
- HttpOnly: Not set (flag absent)
- Secure: Not set (flag absent)
- SameSite: Not configured
- Path: /

Attribute	Value Observed	Risk
HttpOnly	Not Set	High -Cookie accessible via JavaScript (XSS risk)
Secure	Not Set	High - Cookie transmitted over HTTP, vulnerable to sniffing
SameSite	Not Configured	Medium - CSRF attacks possible
Expiry	Session based	Low - Expires on browser close

REFLECTION

The absence of the HttpOnly flag is a significant vulnerability. Without it, any injected JavaScript (via an XSS attack) can directly read the session cookie using `document.cookie`, allowing an attacker to steal the session token and impersonate the victim.

The missing Secure flag means the cookie will also be transmitted over unencrypted HTTP connections, making it susceptible to interception via man-in-the-middle (MITM) attacks.

The lack of a SameSite attribute exposes the application to Cross-Site Request Forgery (CSRF) attacks, where a malicious third-party site can trigger authenticated requests on behalf of a logged-in user.

Mitigation: All session cookies should be configured with **HttpOnly=true**, **Secure=true**, and **SameSite=Strict or Lax**.

Additionally, the application should enforce HTTPS to complement the Secure flag.

Exercise 2: Session Fixation Attack

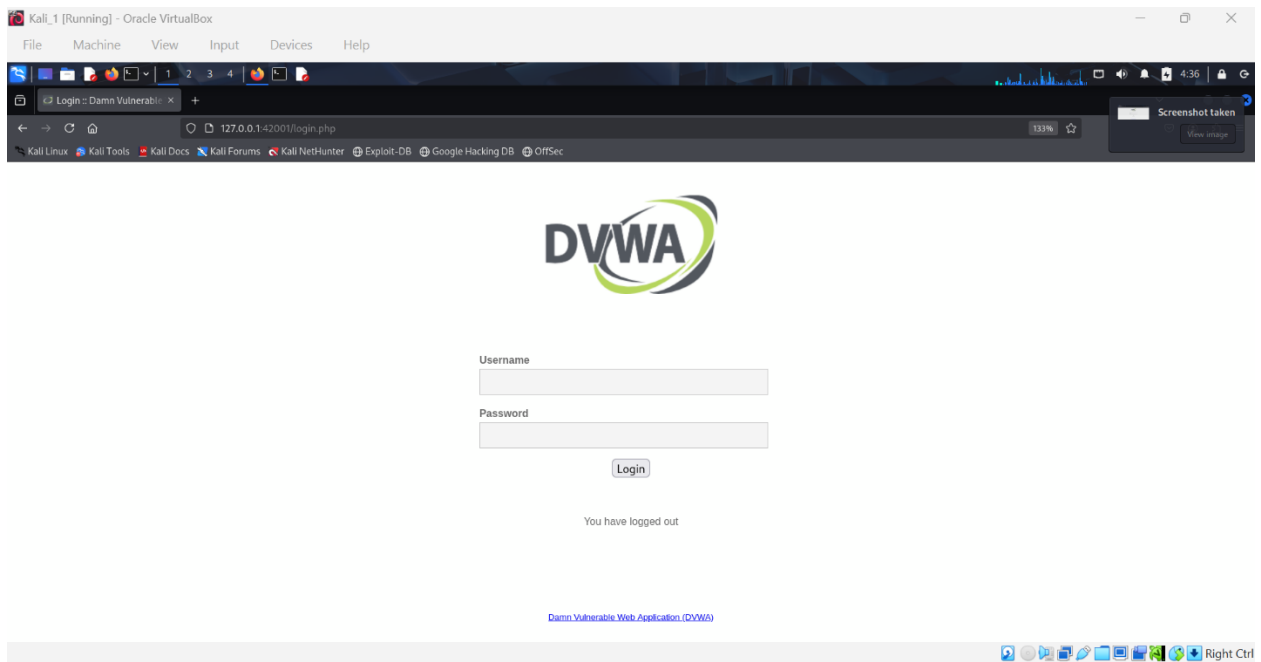


Figure 1 – Open the dvwa login page

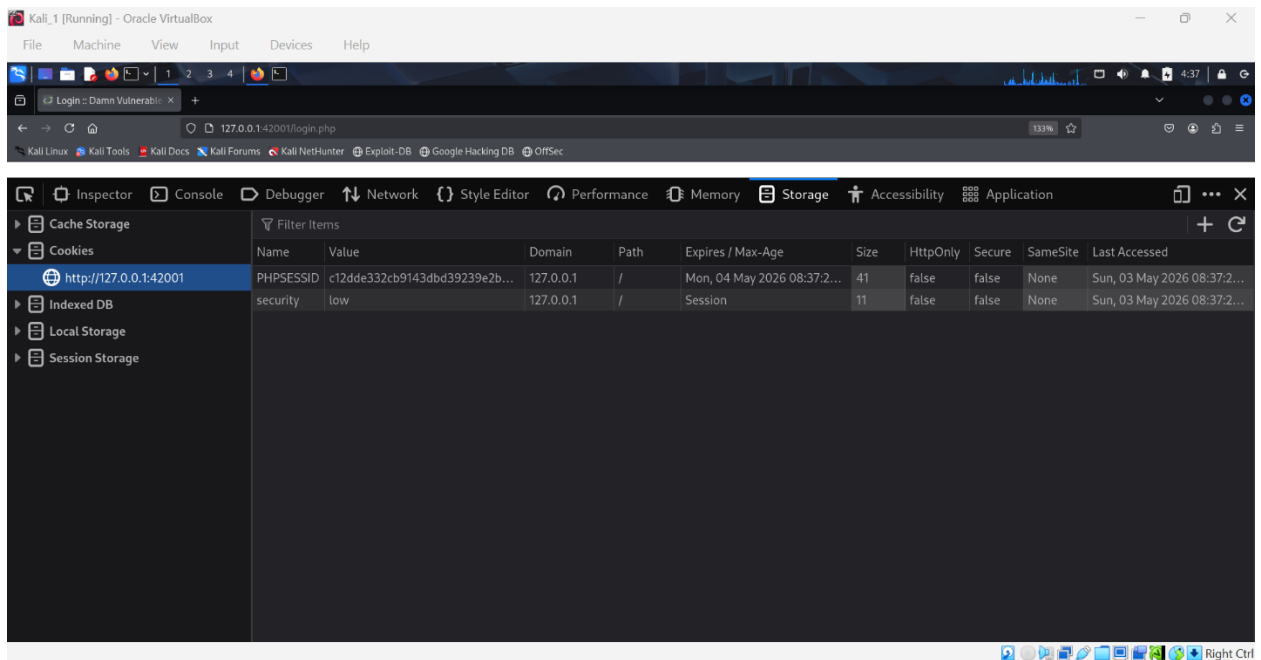


Figure 2- Open the DevTools by pressing F12 on the keyboard, Go to Storage tab, click on Cookie. The Storage tab confirmed the cookie existed but had HttpOnly: false - meaning it was readable but had not yet been issued until page load.

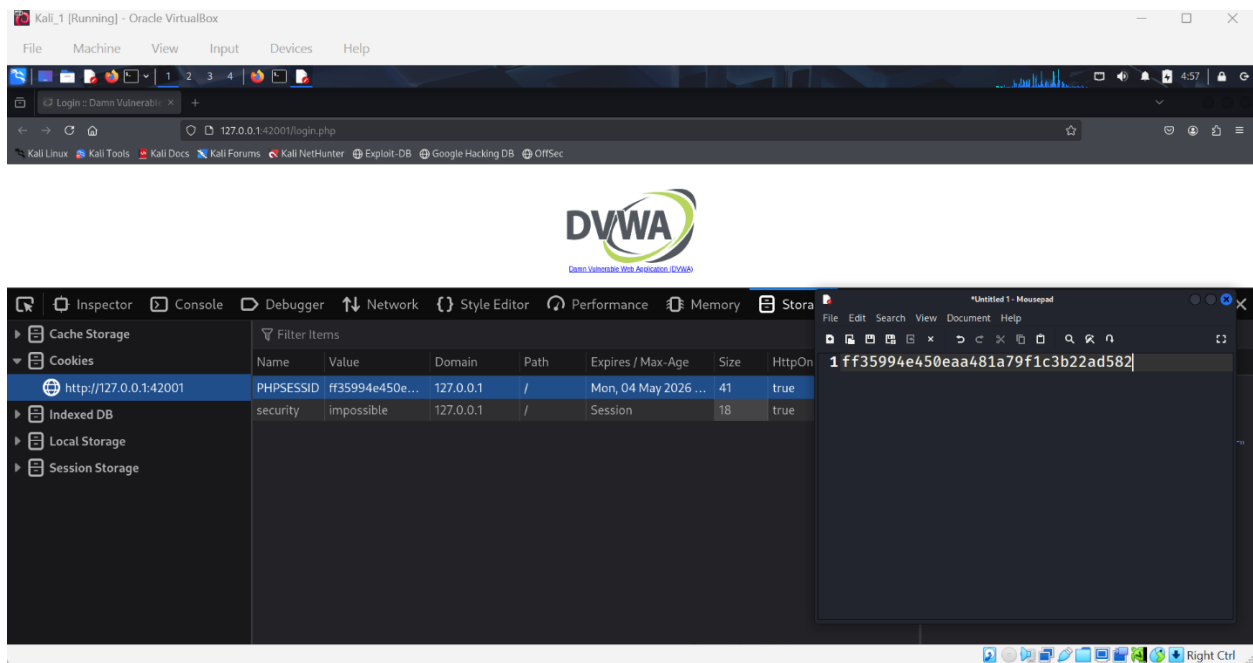


Figure 3 – Click on the PHPSESSID, copy and paste the session Id to the text editor

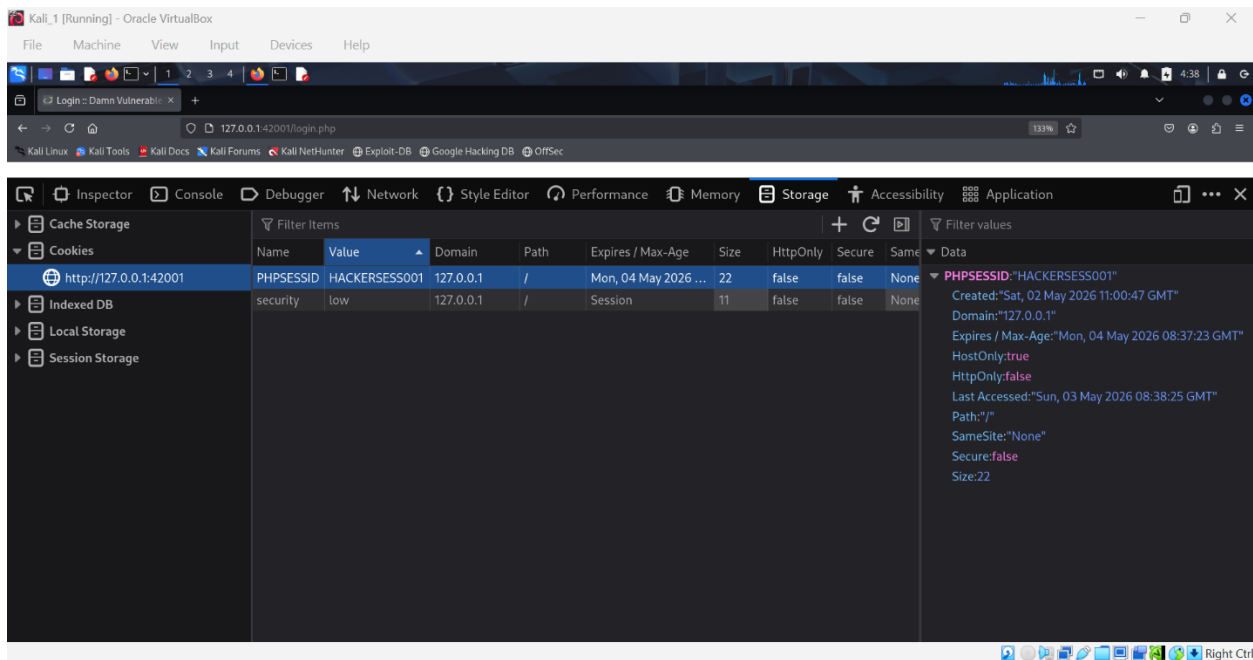


Figure 4 – Change the value on the PHPSESSID to HACKERSESS001. This simulates what an attacker would deliver to a victim via a malicious link or XSS payload. The Storage tab confirmed the new value was accepted by the browser.

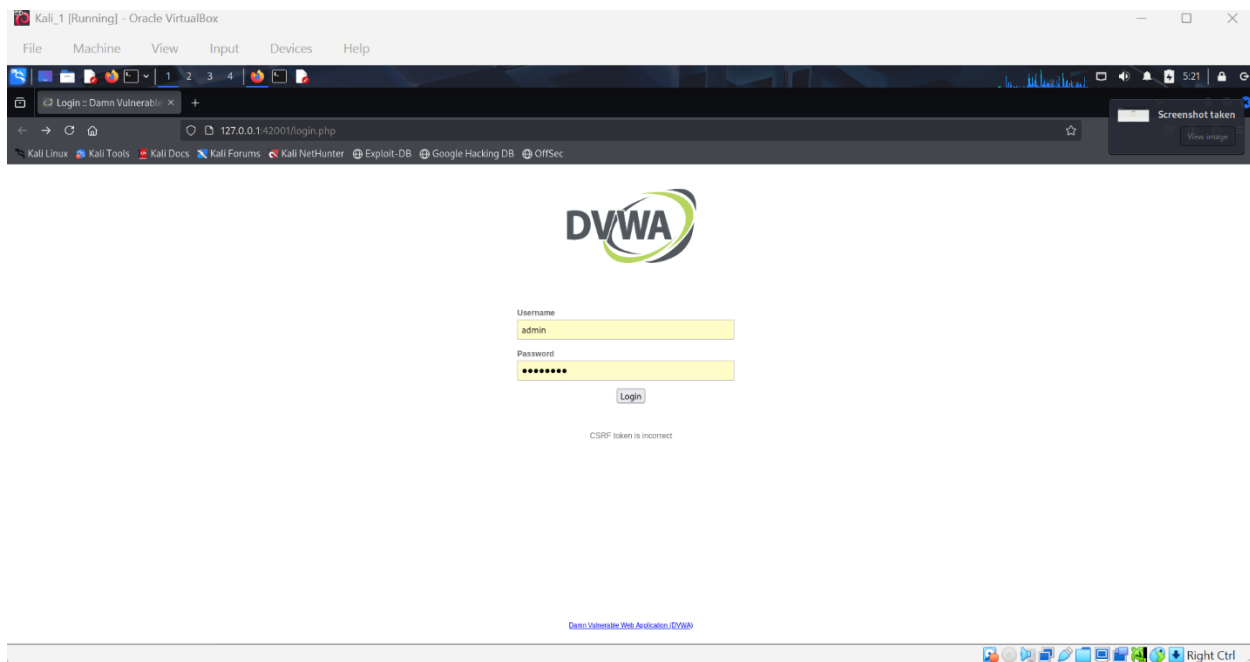


Figure 5 – Login was performed using the default DVWA credentials (admin / password). The login form was submitted normally. After successful authentication, DVWA redirected to the main dashboard (index.php).

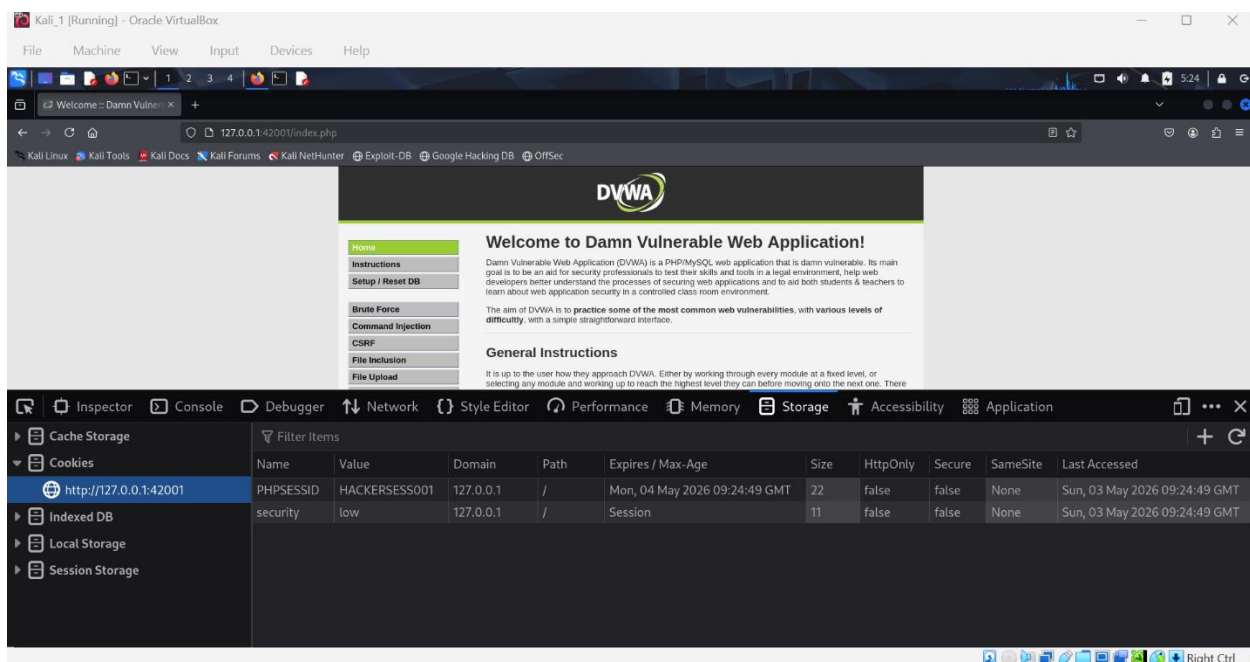


Figure 6 – Returning to the DevTools Storage tab, the PHPSESSID value was inspected again. The value remained: HACKERSESS001. The session ID had not changed. DVWA's login handler did not call session_regenerate_id () at any point. The attacker's planted value was now associated with an authenticated admin session on the server.

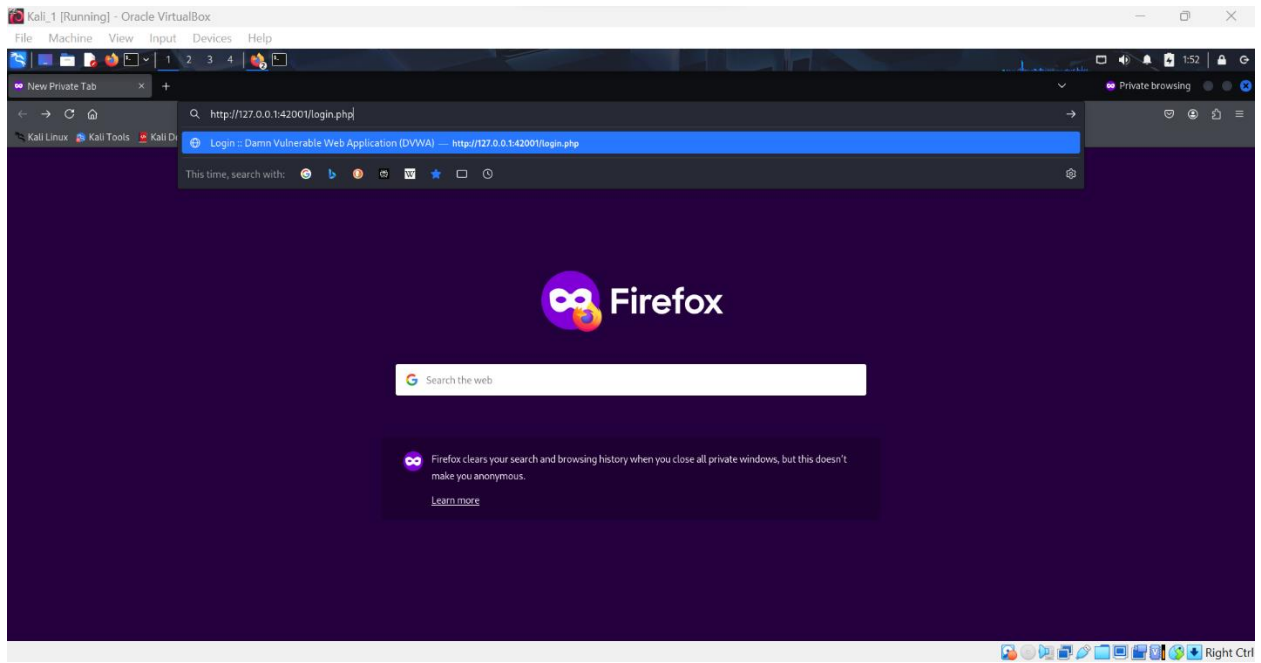


Figure 7 – Using Ctrl +Shift +P to open a private browser and was directed to <http://127.0.0.1:42001/index.php>.

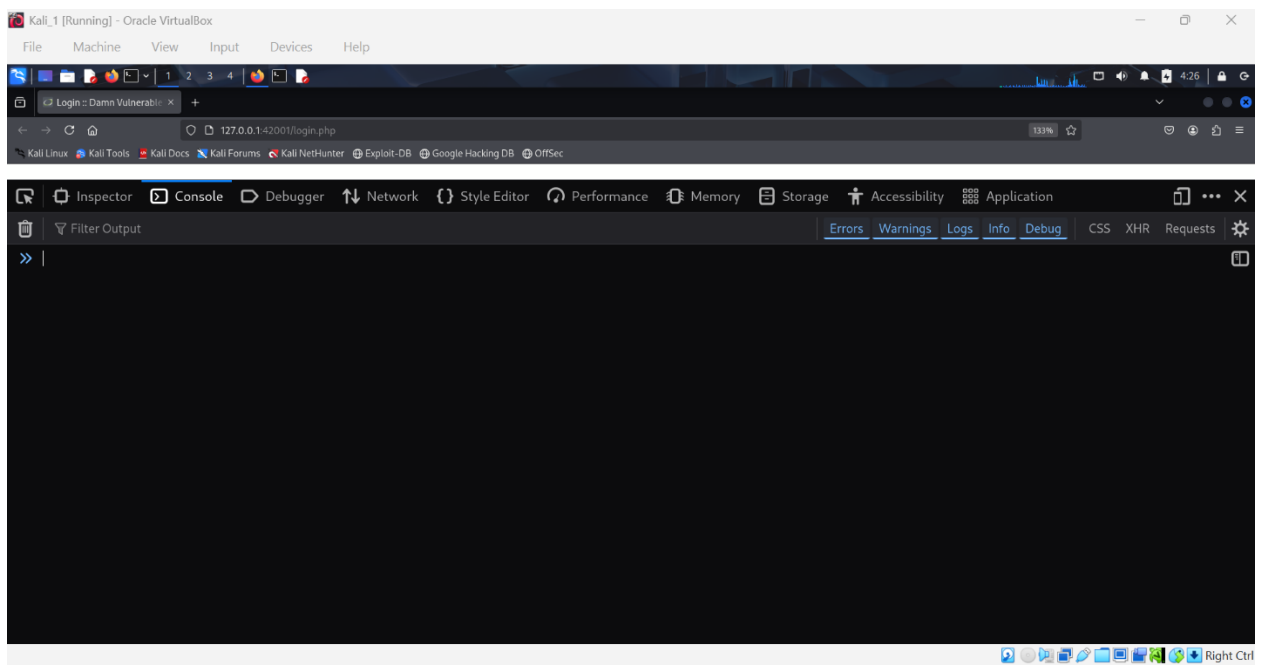


Figure 8 – Go to the Console Tab

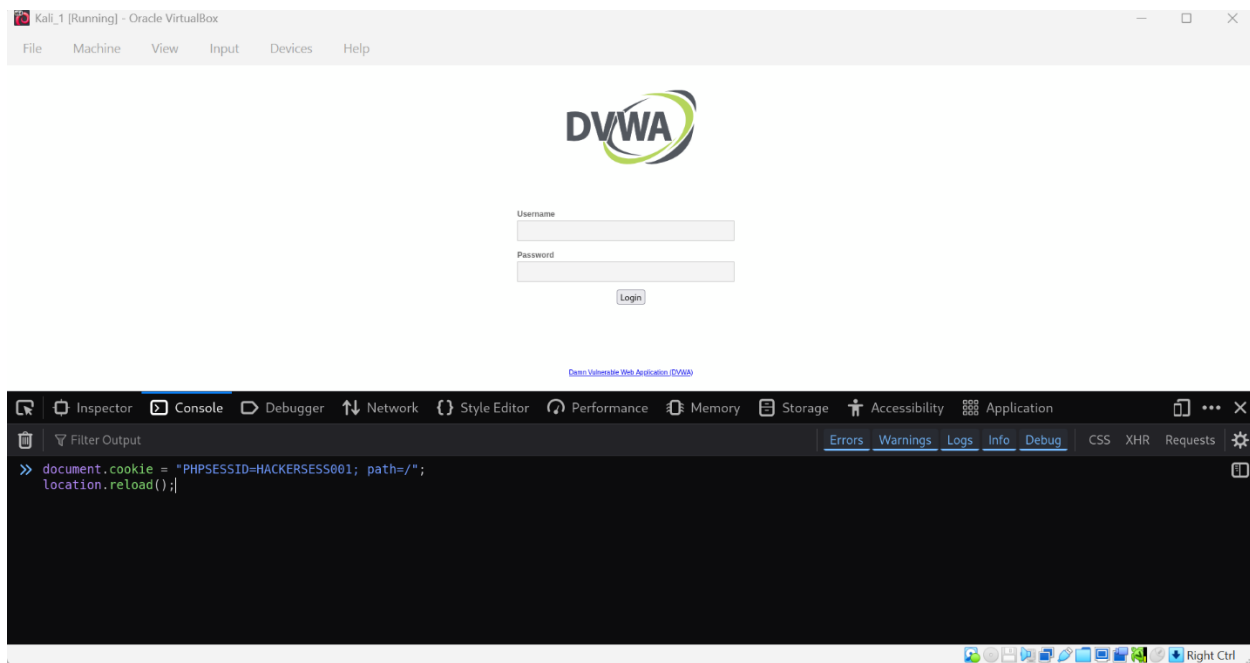


Figure 9 – Enter this command on the console

```
document.cookie = "PHPSESSID = HACKERSESS001; path = /";  
location.reload();
```

The DVWA dashboard loaded fully displaying the authenticated home screen for the admin account without entering any username or password. Full session takeover was confirmed.

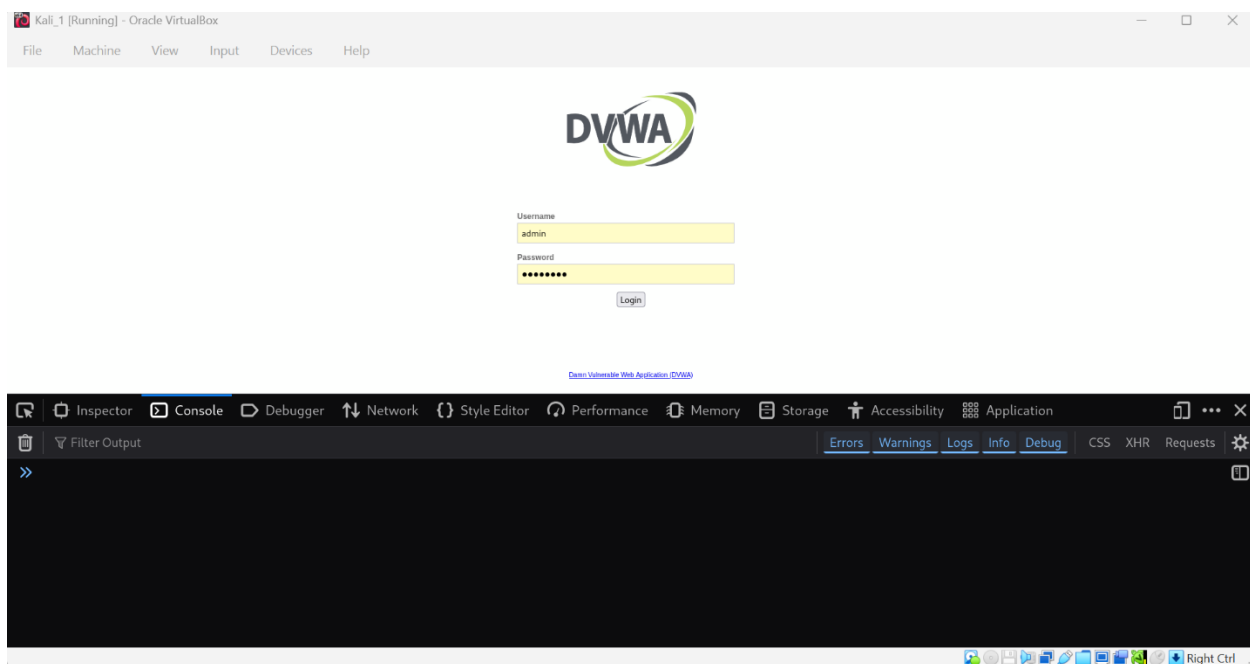


Figure 10 - The DVWA dashboard logged in as admin without entering any credentials.

Analysis of Session Management in DVWA

During the practical exercise conducted on DVWA running at 127.0.0.1:42001, the application demonstrated critically weak session management across its Low, Medium, and High security levels. The following observations were made:

- **Session ID Issuance:** DVWA issues a PHPSESSID cookie upon the first page load at the login screen. This is standard PHP session behavior. However, the application accepted externally set session IDs without any validation meaning a session ID manually planted via the browser's DevTools Storage panel (**HACKERSESS001**) was accepted and honored by the server as a legitimate session.
- **No Session Regeneration on Login:** The most critical flaw observed was that after successful authentication with credentials admin/password, the PHPSESSID value remained unchanged **HACKERSESS001** persisted in the Storage tab post-login. This confirmed that DVWA's login handler does not call `session_regenerate_id()` at any point during the authentication process. The pre-authentication and post-authentication session states shared the same identifier, eliminating any security boundary at the session layer.
- **Insecure Cookie Configuration:** Inspection of the cookie attributes in Firefox DevTools revealed the following:

Attribute	Observed value	Secure value	Risk of Current Setting
HttpOnly	False	True	JavaScript can read/overwrite the cookie enables XSS-based theft
Secure	False	True	Cookie sent over HTTP intercept able via network sniffing
SameSite	None	Strict	Cookie included in all cross-site requests no CSRF protection

MITIGATION

Regenerate session IDs on every privilege level change (login, logout, role change). Also consider binding sessions to the client IP address and User-Agent string to add an additional layer of validation.

Session ID regeneration upon login is the single most critical control against session fixation. Its importance extends beyond just this attack class:

- **Creates a Hard Authentication Boundary**
When `session_regenerate_id(true)` is called immediately after authentication, the server destroys the old session data and issues a new cryptographically random token. Any session ID that existed before login whether server-issued or attacker-planted becomes immediately invalid. The attacker's planted ID no longer maps to any session record on the server.

- **Decouples Anonymous and Authenticated State**

Without regeneration, a session token is merely a tracking identifier with no security meaning it carries no proof of authentication. With regeneration, possession of the post-login token is cryptographic evidence that its holder passed through the authentication gate, because the server only issued that token after verifying credentials.

- **Defeats All Fixation Delivery Vectors Simultaneously**

Whether the attacker delivered the fixed session via a URL parameter, XSS injection, subdomain cookie manipulation, or MITM interception, regeneration defeats all vectors at once. The delivery mechanism becomes irrelevant if the planted ID is unconditionally discarded at login.

- **Minimal Cost, Maximum Impact**

The remediation requires a single line of code added to the login handler:

`session_regenerate_id(true);` // Called immediately after credential verification

The true parameter is essential — it deletes the old session file server-side, not just on the client. Without it, the old session record persists on the server and could still be exploited if the attacker obtained it by other means.

Control	Implementation	Defeats
Regenerate session on login	<code>session_regenerate_id(true)</code> called immediately after auth	Session fixation primary fix
HttpOnly cookie flag	<code>session_set_cookie_params(['httponly' => true])</code>	XSS-based session theft
Secure cookie flag	<code>session_set_cookie_params(['secure' => true])</code>	Network sniffing attacks
SameSite=Strict	<code>session_set_cookie_params(['samesite' => 'Strict'])</code>	CSRF and cross-site attacks
Disable URL-based session IDs	<code>ini_set('session.use_only_cookies', 1)</code>	URL-delivered fixation
Session timeout	Absolute and idle timeouts enforced server-side	Persistent session attacks
Session binding	Bind session to IP + User-Agent on login	Hijack from different client
Destroy session on logout	<code>session_destroy()</code> + <code>unset(\$_SESSION)</code>	Session reuse after logout

Exercise 3: Session Hijacking Using Burp Suite

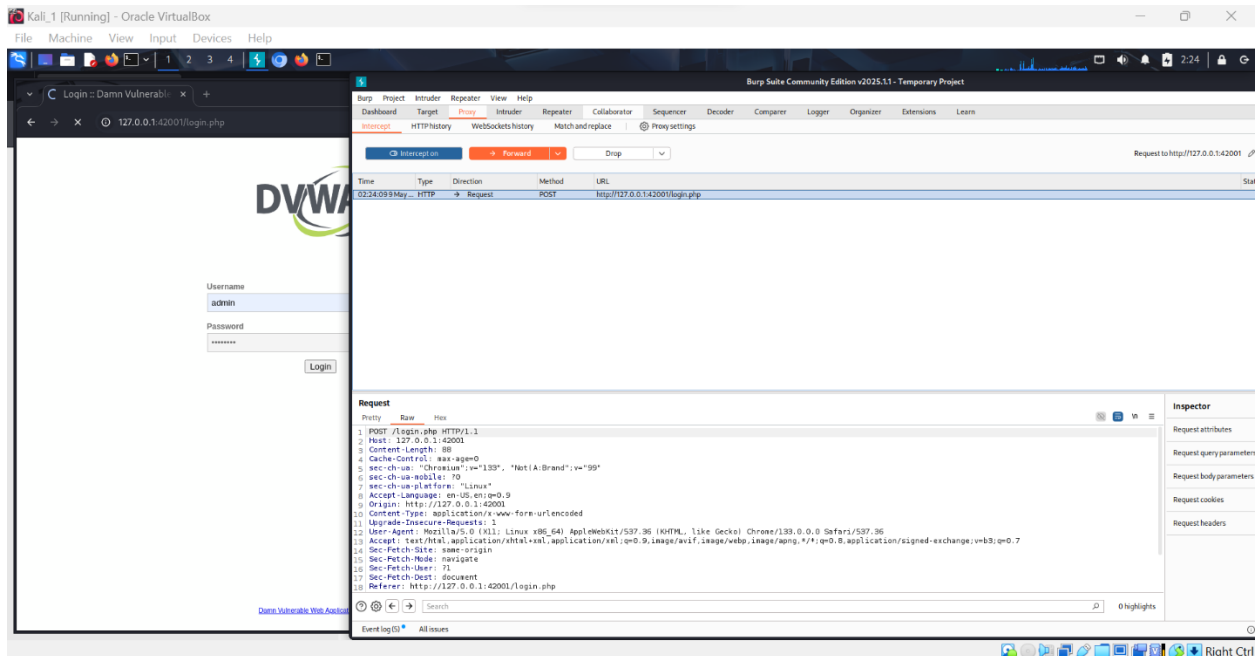


Figure 11 - In Burp Suite, Proxy > Intercept was opened and the toggle was clicked until it read Intercept is on. All requests from Firefox are now held in Burp until manually forwarded. In Firefox, the DVWA login page was opened at <http://127.0.0.1:42001/login.php>. Credentials admin / password were entered and the Login button was clicked. The browser froze confirming Burp had intercepted the POST request before it reached the server.

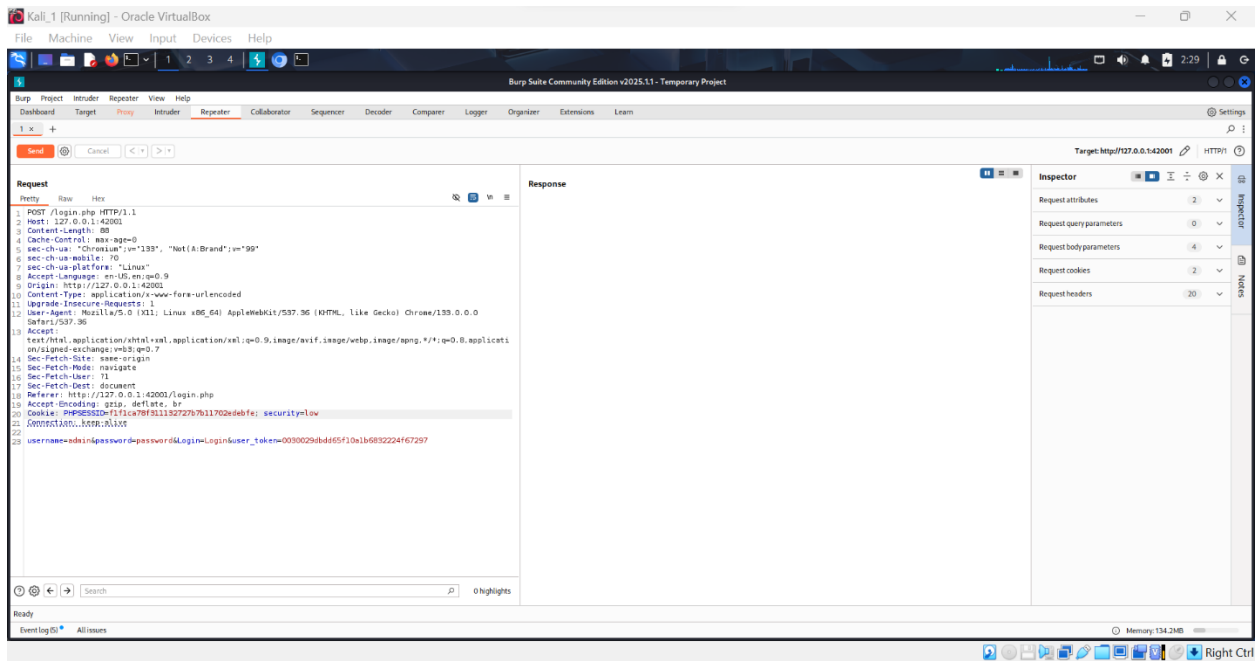


Figure 12 - In the Burp Response tab, the Cookie header line was clicked and the PHPSESSID value was changed from the server-issued value to the attacker-controlled value: HACKERSESS001. This simulates an attacker injecting a known, predetermined session token into the authentication request.

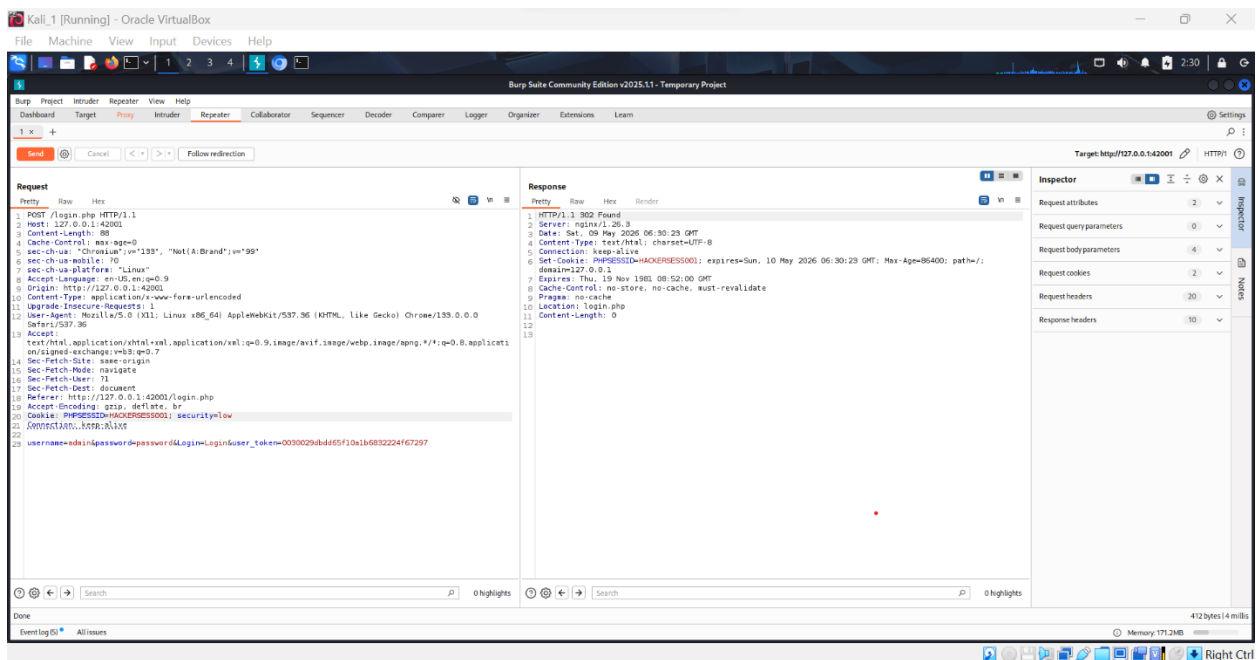


Figure 13 - In the Repeater tab, the Cookie header was confirmed to contain PHPSESSID=HACKERSESS001. The Send button was clicked. The response panel returned HTTP 200 OK with the full DVWA dashboard HTML confirming the session was valid and authenticated.

